

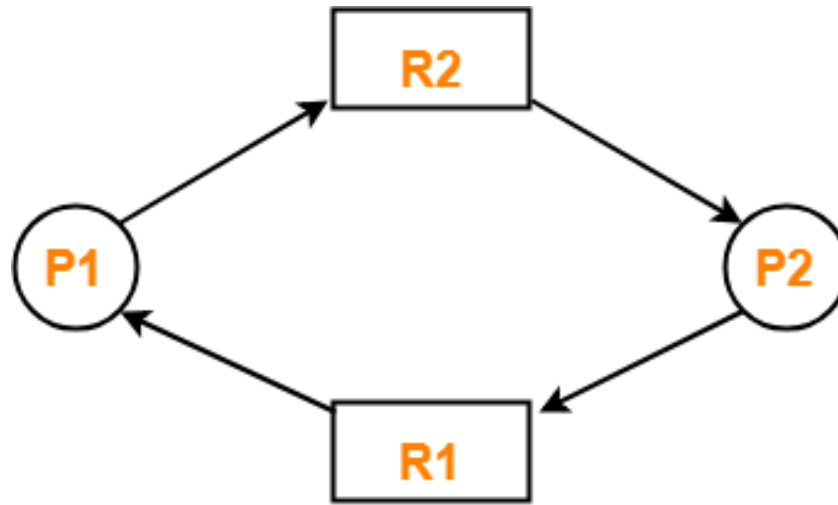
Deadlock Handling

Unit IV

Deadlock in OS

Deadlock is a situation where-

The execution of two or more processes is blocked because each process holds some resource and waits for another resource held by some other process.



Example of a deadlock

Here

- Process P1 holds resource R1 and waits for resource R2 which is held by process P2.
- Process P2 holds resource R2 and waits for resource R1 which is held by process P1.
- None of the two processes can complete and release their resource.
- Thus, both the processes keep waiting infinitely.

Starvation vs Deadlock?

❖ The abstract sequence of events required to use a resource is given below.

1. Request the resource.
2. Use the resource.
3. Release the resource

Conditions For Deadlock

There are following 4 necessary conditions for the occurrence of deadlock-

1. Mutual Exclusion
2. Hold and Wait
3. No preemption
4. Circular wait

Livelock vs Deadlock

- **Deadlock:** Processes **wait forever** doing nothing.
- **Livelock:** Processes **move forever** doing nothing.

1. Mutual Exclusion-

By this condition,

- There must exist at least one resource in the system which can be used by only one process at a time.
- If there exists no such resource, then deadlock will never occur.
- Printer is an example of a resource that can be used by only one process at a time.

2. Hold and Wait-

By this condition,

- There must exist a process which holds some resource and waits for another resource held by some other process.

3. No Preemption-

By this condition,

Once the resource has been allocated to the process, it can not be preempted.

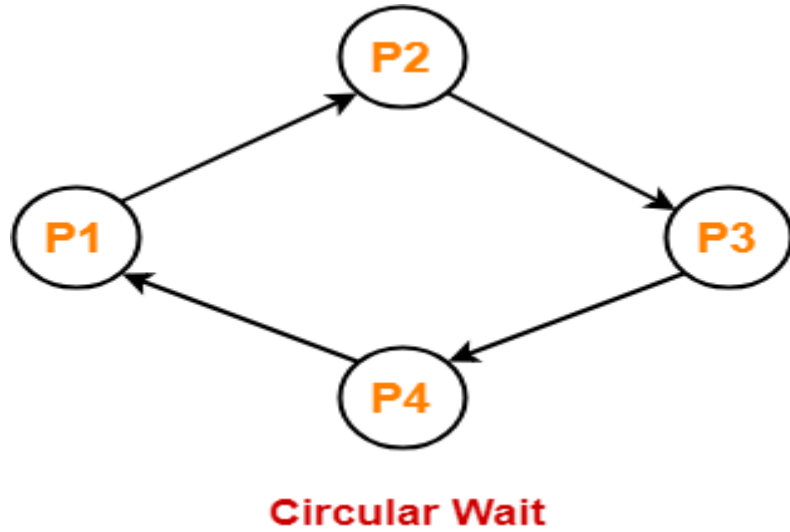
It means resource can not be snatched forcefully from one process and given to the other process.

The process must release the resource voluntarily by itself.

4. Circular Wait-

By this condition,

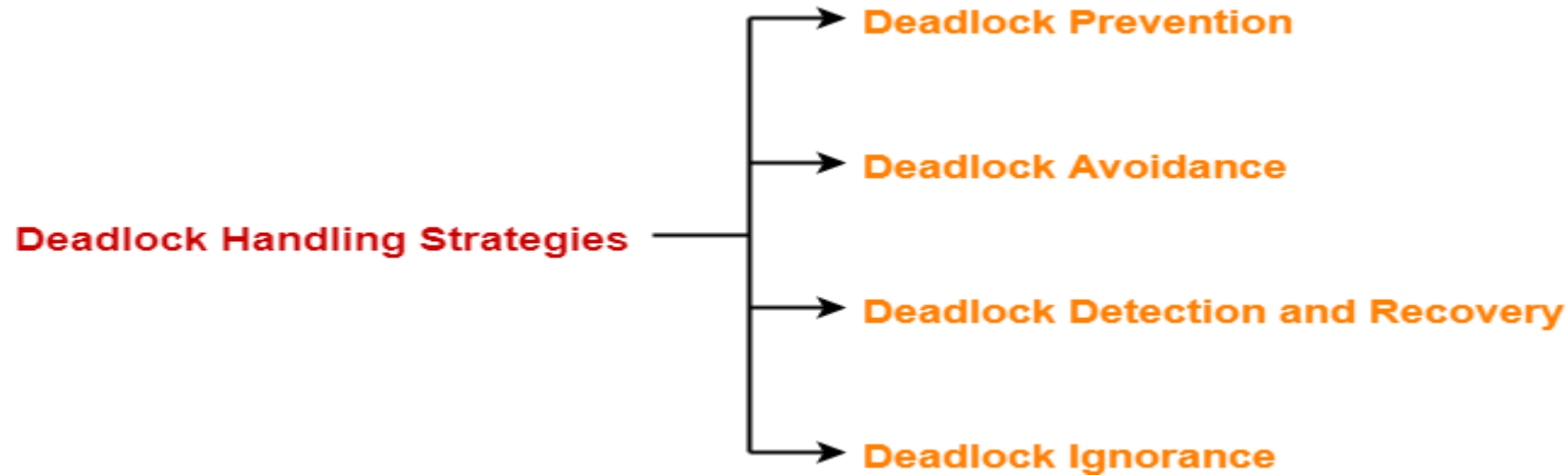
All the processes must wait for the resource in a cyclic manner where the last process waits for the resource held by the first process.



Note-

- All these 4 conditions must hold simultaneously for the occurrence of deadlock.
- If any of these conditions fail, then the system can be ensured deadlock free.

Deadlock Handling



- Ensure that the system will **never** enter a deadlock state:
 - Deadlock prevention
 - Deadlock avoidance
- Allow the system to enter a deadlock state and then recover
- Ignore the problem and pretend that deadlocks never occur in the system.

Deadlock Prevention-

- This strategy involves designing a system that violates one of the four necessary conditions required for the occurrence of deadlock.
- This ensures that the system remains free from the deadlock.

The various conditions of deadlock occurrence may be violated as:

1. Mutual Exclusion-

To violate this condition, all the system resources must be such that they can be used in a shareable mode. In a system, there are always some resources which are mutually exclusive by nature. So, this condition can not be violated.

2. No Preemption-

- This condition can be violated by **forceful preemption**.
- Consider a process is holding some resources and request other resources that can not be immediately allocated to it.
- Then, by forcefully preempting the currently held resources, the condition can be violated.

A process is allowed to forcefully preempt the resources possessed by some other process only if-

It is a high priority process or a system process.

The victim process is in the waiting state.

3. Circular Wait-

- This condition can be violated by not allowing the processes to wait for resources in a cyclic manner.
- To violate this condition, the following approach is followed-

Approach-

- A natural number is assigned to every resource.
- Each process is allowed to request for the resources either in only increasing or only decreasing order of the resource number.
- In case increasing order is followed, if a process requires a lesser number resource, then it must release all the resources having larger number and vice versa.
- This approach is the most practical approach and implementable.
- However, this approach may cause starvation but will never lead to deadlock.

4. Hold and Wait-

This condition can be violated in the following ways-

Approach-01: (Conservative Approach)

In this approach,

- A process has to first request for all the resources it requires for execution.
- Once it has acquired all the resources, only then it can start its execution.
- This approach ensures that the process does not hold some resources and wait for other resources.
- This approach is easy and is deadlock independent.

Drawbacks-

The drawbacks of this approach are-

- It is less efficient.
- It is not implementable since it is not possible to predict in advance which resources will be required during execution.

Approach-02: (Do not hold)

In this approach,

- A process is allowed to acquire the resources it desires at the current moment.
- After acquiring the resources, it start its execution.
- Now before making any new request, it has to compulsorily release all the resources that it holds currently.
- This approach is efficient and implementable.

Approach-03 (Wait timeouts):

In this approach,

- A timer is set after the process acquires any resource.
- After the timer expires, a process has to compulsorily release the resource.

GATE Question

Q: Consider the following policies for preventing deadlock in a system with mutually exclusive resources.

- I. Process should acquire all their resources at the beginning of execution. If any resource is not available, all resources acquired so far are released.
- II. The resources are numbered uniquely, and processes are allowed to request for resources only in increasing resource numbers
- III. The resources are numbered uniquely, and processes are allowed to request for resources only in decreasing resource numbers
- IV. The resources are numbered uniquely. A processes is allowed to request for resources only for a resource with resource number larger than its currently held resources

Which of the above policies can be used for preventing deadlocks?

- A. Any one of (I) and (III) but not (II) or (IV)
- B. Any one of (I), (III) and (IV) but not (II)
- C. Any one of (II) and (III) but not (I) or (IV)
- D. Any one of (I), (II), (III) and (IV)**

Q: A system is having 3 User processes, each requiring 2 units of resources “R”. The minimum number of units of R such that no deadlock will occur?

A) 3 B) 5 C) 4 D) 6

General Solution

In order to answer such questions, find the maximum number of resources that when allocated can cause a Deadlock and then add “1” to find the minimum number of resources , that will make the system deadlock free.

Example

Process	P1	P2	P3
Requirement	2	3	4

Maximum Resources allocated so that deadlock can occur :

$$1+2+3 = 6$$

Minimum Resources so that deadlock can not occur :

$$6+1 = 7$$

Important concepts to solve problems on deadlock

Consider there are n processes in the system $P_1, P_2, P_3, \dots, P_n$ where-

- Process P_1 requires x_1 units of resource R
- Process P_2 requires x_2 units of resource R
- Process P_3 requires x_3 units of resource R and so on.

In worst case,

The number of units that each process holds = One less than its maximum demand

So,

Process P_1 holds $x_1 - 1$ units of resource R

Process P_2 holds $x_2 - 1$ units of resource R

Process P_3 holds $x_3 - 1$ units of resource R and so on.

Now,

Had there been one more unit of resource R in the system, system could be ensured deadlock free.

This is because that unit would be allocated to one of the processes and it would get execute and then release its units.

From here, we have-

Maximum Number Of Units That Ensures Deadlock-

Maximum number of units of resource R that ensures deadlock

$$= (x_1 - 1) + (x_2 - 1) + (x_3 - 1) + \dots + (x_n - 1)$$

$$= (x_1 + x_2 + x_3 + \dots + x_n) - n$$

$$= \sum x_i - n$$

= Sum of max needs of all n processes – n

Minimum Number Of Units That Ensures No Deadlock-

Minimum number of units of resource R that ensures no deadlock

= One more than maximum number of units of resource R that ensures deadlock

$$= (\sum x_i - n) + 1$$

PRACTICE PROBLEMS BASED ON DEADLOCK IN OS-

Problem-01:

A system is having 3 user processes each requiring 2 units of resource R. The minimum number of units of R such that no deadlock will occur?

Solution-

In worst case,

The number of units that each process holds = One less than its maximum demand

So,

Process P1 holds 1 unit of resource R

Process P2 holds 1 unit of resource R

Process P3 holds 1 unit of resource R

Thus,

Maximum number of units of resource R that ensures deadlock = $1 + 1 + 1 = 3$

Minimum number of units of resource R that ensures no deadlock = $3 + 1 = 4$

Problem-02:

A system is having 10 user processes each requiring 3 units of resource R. The minimum number of units of R such that no deadlock will occur _____?

Solution-

In worst case,

The number of units that each process holds = One less than its maximum demand

Thus,

Maximum number of units of resource R that ensures deadlock = $10 \times 2 = 20$

Minimum number of units of resource R that ensures no deadlock = $20 + 1 = 21$

Problem-03:

A system is having 3 user processes P1, P2 and P3 where P1 requires 2 units of resource R, P2 requires 3 units of resource R, P3 requires 4 units of resource R. The minimum number of units of R that ensures no deadlock is _____?

Solution- In worst case, The number of units that each process holds = One less than its maximum demand.

Maximum number of units of resource R that ensures deadlock = $1 + 2 + 3 = 6$

Minimum number of units of resource R that ensures no deadlock = $6 + 1 = 7$

Problem-04:

If there are 6 units of resource R in the system and each process in the system requires 2 units of resource R, then how many processes can be present at maximum so that no deadlock will occur?

Solution-

In worst case,

The number of units that each process holds = One less than its maximum demand

So,

Process P1 holds 1 unit of resource R

Process P2 holds 1 unit of resource R

Process P3 holds 1 unit of resource R

Process P4 holds 1 unit of resource R

Process P5 holds 1 unit of resource R

Process P6 holds 1 unit of resource R

Thus,

Minimum number of processes that ensures deadlock = 6

Maximum number of processes that ensures no deadlock = $6 - 1 = 5$

Problem-05:

If there are 100 units of resource R in the system and each process in the system requires 2 units of resource R, then how many processes can be present at maximum so that no deadlock will occur?

Solution- In worst case, The number of units that each process holds = One less than its maximum demand
So,

Process P1 holds 1 unit of resource R

Process P2 holds 1 unit of resource R

Process P3 holds 1 unit of resource R and so on.

Process P100 holds 1 unit of resource R

Minimum number of processes that ensures deadlock = 100

Maximum number of processes that ensures no deadlock = $100 - 1 = 99$

Problem-06:

Consider a system having **m resources** of the same type. These resources are shared by 3 **processes A, B and C** which have peak demands of **3, 4 and 6** respectively. For what value of **m**, deadlock will not occur?

a) 7 b) 9 c) 10 d) 13

Solution- In worst case,

The number of units that each process holds = One less than its maximum demand

Process A holds 2 units of resource R

Process B holds 3 units of resource R

Process C holds 5 units of resource R

Thus,

Maximum number of units of resource R that ensures deadlock = $2 + 3 + 5 = 10$

Minimum number of units of resource R that ensures no deadlock = $10 + 1 = 11$

Resource Allocation Graph (RAG)

Resource Allocation Graph (RAG) is a graph that represents the state of a system pictorially.

It gives complete information about the state of a system such as:

- How many processes exist in the system?
- How many instances of each resource type exist?
- How many instances of each resource type are allocated?
- How many instances of each resource type are still available?
- How many instances of each resource type are held by each process?
- How many instances of each resource type does each process need for execution?

Components Of RAG-

There are two major components of a Resource Allocation Graph-

1.Vertices

2.Edges

Vertices-

There are following types of vertices in a Resource Allocation Graph-

Process Vertices

Resource Vertices

Process Vertices-

Process vertices represent the processes.

They are drawn as a circle by mentioning the name of process inside the circle.

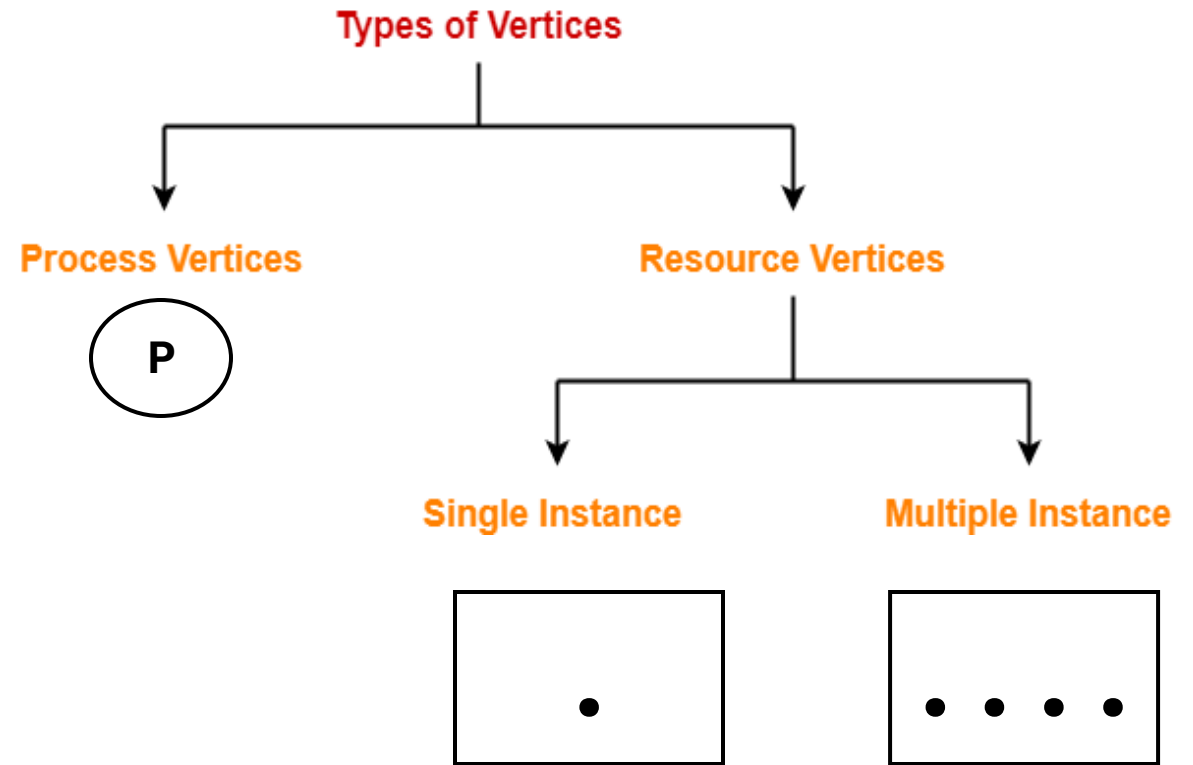
Resource Vertices-

Resource vertices represent the resources.

Depending on the number of instances that exists in the system, resource vertices may be single instance or multiple instance.

They are drawn as a rectangle by mentioning the dots inside the rectangle.

The number of dots inside the rectangle indicates the number of instances of that resource existing in the system.



Edges-

There are two types of edges in a Resource Allocation Graph-

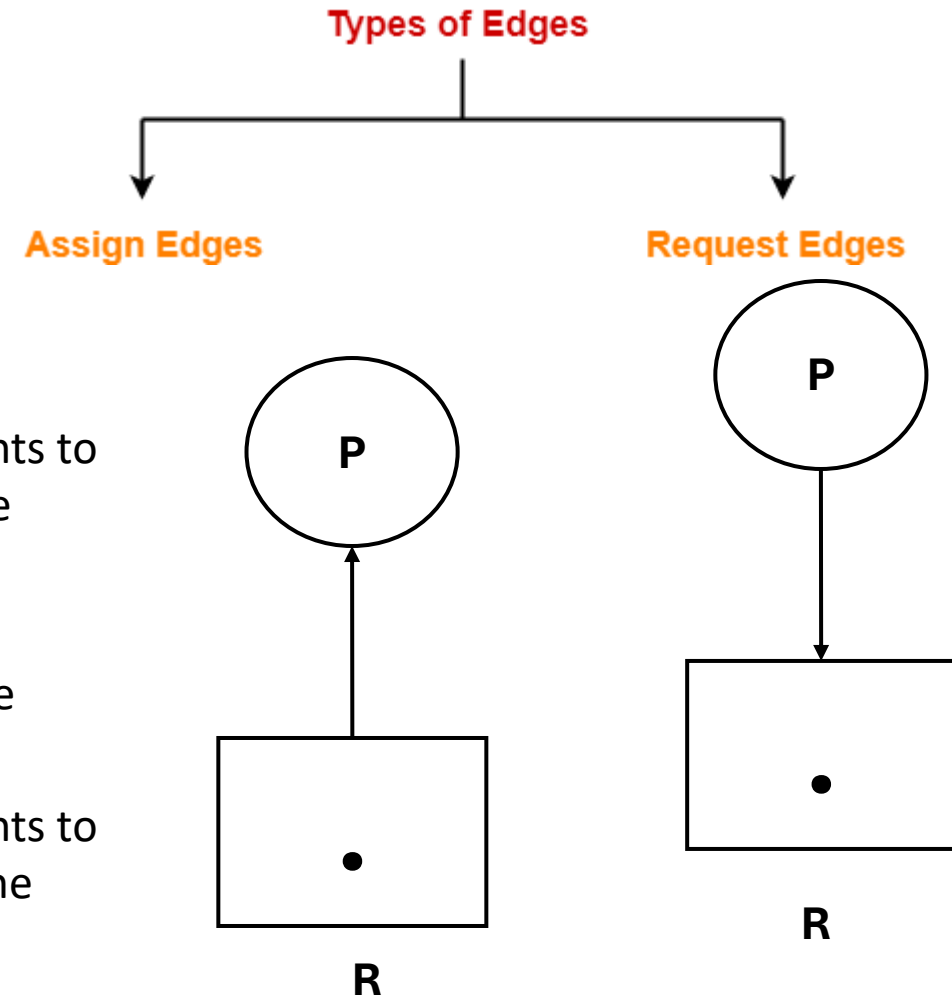
1. Assign Edges
2. Request Edges

Assign Edges-

- Assign edges represent the assignment of resources to the processes.
- They are drawn as an arrow where the head of the arrow points to the process and tail of the process points to the instance of the resource.

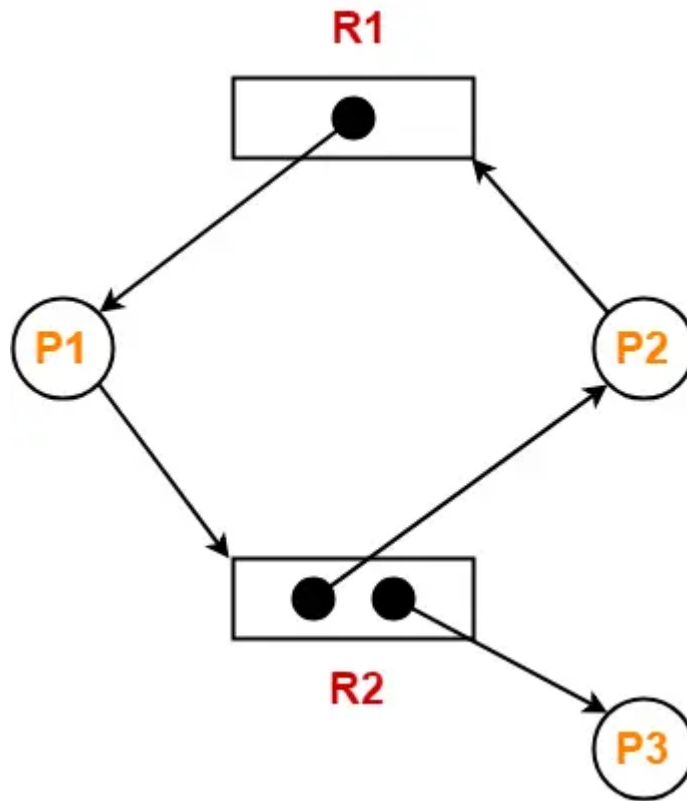
Request Edges-

- Request edges represent the waiting state of processes for the resources.
- They are drawn as an arrow where the head of the arrow points to the instance of the resource and tail of the process points to the process.
- If a process requires 'n' instances of a resource type, then 'n' assign edges will be drawn.



Example Of RAG-

The following diagram represents a Resource Allocation Graph:



It gives the following information-

- There exist three processes in the system namely P1, P2 and P3.
- There exist two resources in the system namely R1 and R2.
- There exists a single instance of resource R1 and two instances of resource R2.
- Process P1 holds one instance of resource R1 and is waiting for an instance of resource R2.
- Process P2 holds one instance of resource R2 and is waiting for an instance of resource R1.
- Process P3 holds one instance of resource R2 and is not waiting for anything.

Resource Allocation Graph(RAG) contd...

Using Resource Allocation Graph, it can be easily detected whether system is in a **Deadlock** state or not.

The rules are-

Rule-01:

In a Resource Allocation Graph where all the resources are single instance,

- If a cycle is being formed, then system is in a deadlock state.
- If no cycle is being formed, then system is not in a deadlock state.

Rule-02:

In a Resource Allocation Graph where all the resources are **NOT** single instance,

- If a cycle is being formed, then system may be in a deadlock state.
- Banker's Algorithm** is applied to confirm whether system is in a deadlock state or not.
- If no cycle is being formed, then system is not in a deadlock state.
- Presence of a cycle is a necessary but not a sufficient condition for the occurrence of deadlock.

PRACTICE PROBLEMS BASED ON DETECTING DEADLOCK USING RAG

Problem-01:

Consider the resource allocation graph in the figure-

Find if the system is in a deadlock state otherwise find a safe sequence.

Solution-

Method-01:

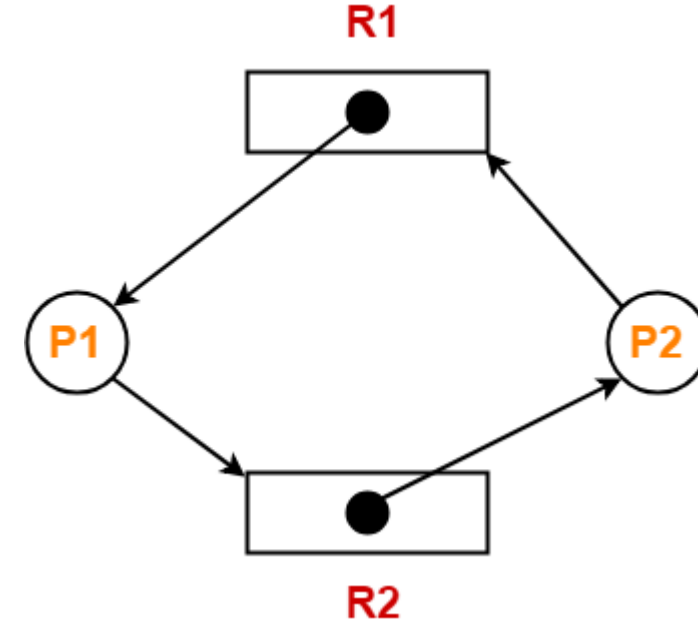
- The given resource allocation graph is single instance with a cycle contained in it.
- Thus, the system is definitely in a **deadlock state**.

Method-02:

Using the given resource allocation graph, we have-

	Allocation		Need	
	R1	R2	R1	R2
Process P1	1	0	0	1
Process P2	0	1	1	0

Available = [R1 R2] = [0 0]

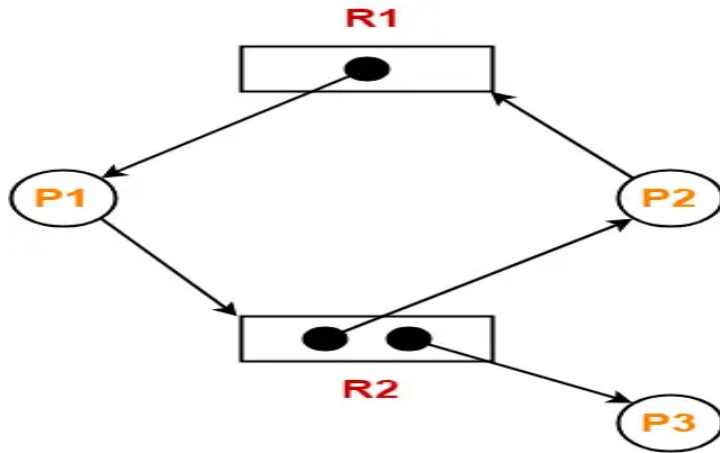


Now,

- There are no instances available currently and both the processes require a resource to execute.
- Therefore, none of the process can be executed and both keeps waiting infinitely.
- Thus, the system is in a **deadlock state**.

Problem-02:

Consider the resource allocation graph in the figure-



Find if the system is in a deadlock state otherwise find a safe sequence.

Solution-

- The given resource allocation graph is multi-instance with a cycle contained in it.
- So, the system may or may not be in a deadlock state.
- Using the given resource allocation graph, we have-

	Allocation		Need	
	R1	R2	R1	R2
Process P1	1	0	0	1
Process P2	0	1	1	0
Process P3	0	1	0	0

Available = [R1 R2] = [0 0]

Step-01:

- Since process P3 does not need any resource, so it executes.
- After execution, process P3 release its resources.

Then, Available = [0 0] + [0 1] = [0 1]

Step-02:

- With the instances available currently, only the requirement of the process P1 can be satisfied.
- So, process P1 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then- Available = [0 1] + [1 0] = [1 1]

Step-03:

With the instances available currently, the requirement of the process P2 can be satisfied. So, process P2 is allocated the requested resources. It completes its execution and then free up the instances of resources held by it.

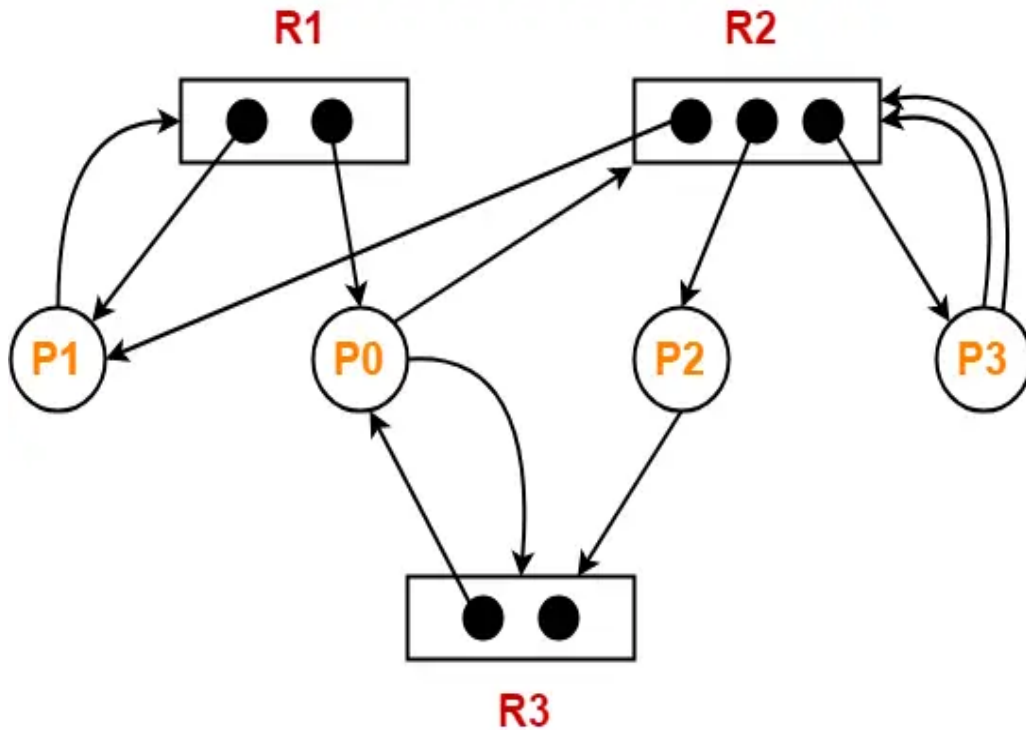
Then- Available = [1 1] + [0 1] = [1 2]

Thus,

There exists a safe sequence P3, P1, P2 in which all the processes can be executed. So, the system is in a safe state.

Problem-03:

Consider the resource allocation graph in the figure



Find if the system is in a deadlock state otherwise find a safe sequence.

Solution-

- The given resource allocation graph is multi-instance with a cycle contained in it.
 - So, the system may or may not be in a deadlock state.
- Using the given resource allocation graph, we have-

	Allocation			Need		
	R1	R2	R3	R1	R2	R3
Process P0	1	0	1	0	1	1
Process P1	1	1	0	1	0	0
Process P2	0	1	0	0	0	1
Process P3	0	1	0	0	2	0

Available = [R1 R2 R3] = [0 0 1]

Step-01:

- With the instances available currently, only the requirement of the process P2 can be satisfied.
- So, process P2 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then- Available

$$= [0 \ 0 \ 1] + [0 \ 1 \ 0]$$

$$= [0 \ 1 \ 1]$$

Step-02:

- With the instances available currently, only the requirement of the process P0 can be satisfied.
- So, process P0 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then- Available

$$= [0 \ 1 \ 1] + [1 \ 0 \ 1]$$

$$= [1 \ 1 \ 2]$$

Step-03:

- With the instances available currently, only the requirement of the process P1 can be satisfied.
- So, process P1 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then- Available

$$= [1 \ 1 \ 2] + [1 \ 1 \ 0] = [2 \ 2 \ 2]$$

Step-04:

- With the instances available currently, the requirement of the process P3 can be satisfied.
- So, process P3 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then- Available

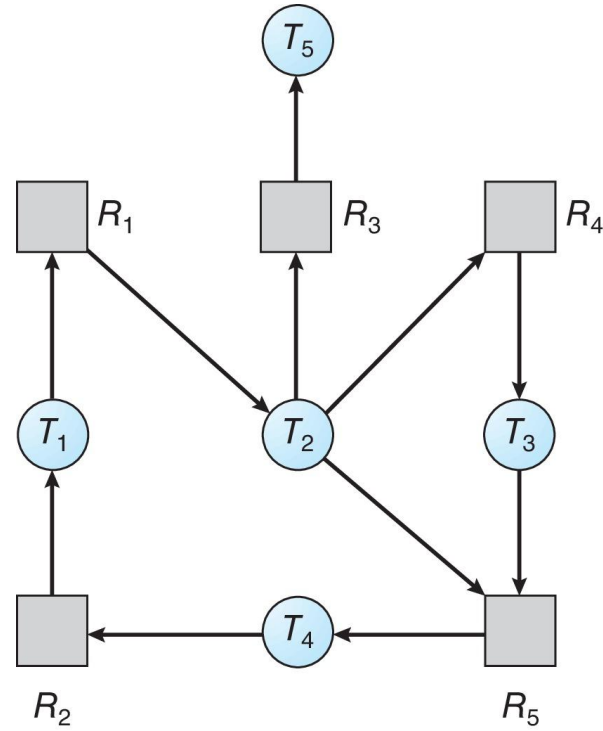
$$= [2 \ 2 \ 2] + [0 \ 1 \ 0] = [2 \ 3 \ 2]$$

Thus,

There exists a safe sequence **P2, P0, P1, P3** in which all the processes can be executed.

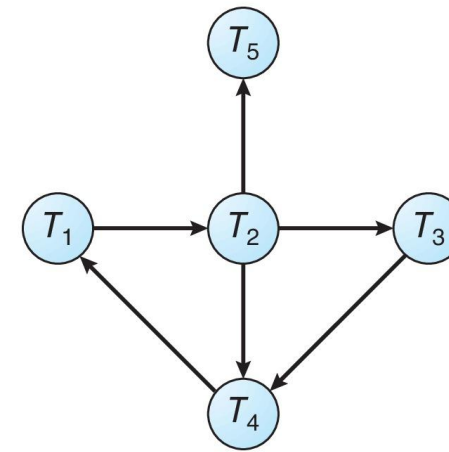
So, the system is in a safe state.

Resource-Allocation Graph and Wait-for Graph



(a)

Resource-Allocation Graph



(b)

Corresponding wait-for graph

Banker's Algorithm-

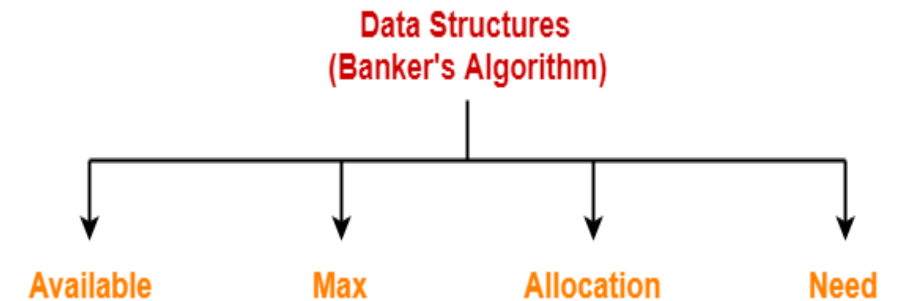
- Banker's Algorithm is a deadlock avoidance strategy.
- It is called so because it is used in banking systems to decide whether a loan can be granted or not.

Prerequisite-

Banker's Algorithm requires: Whenever a new process is created, it specifies the maximum number of instances of each resource type that it exactly needs.

Data Structures Used-

To implement banker's algorithm, following four data structures are used-



Available: A vector $\text{Available}[R1] = K$

It means K instances of resource type R1 are currently available.

Max: $\text{Max}[P1][R1] = K$

It means process P1 is allowed to ask for maximum K instances of resource type R1.

Allocation: $\text{Allocation}[P1][R1] = K$

It means K instances of resource type R1 have been allocated to the process P1.

Need: $\text{Need}[P1][R1] = K$

It means process P1 requires K more instances of resource type R1 for execution.

❖ Banker's algorithm includes safety algorithm and resource request algorithm

8.6.3.1 Safety Algorithm

We can now present the algorithm for finding out whether or not a system is in a safe state. This algorithm can be described as follows:

1. Let *Work* and *Finish* be vectors of length *m* and *n*, respectively. Initialize *Work* = *Available* and *Finish*[*i*] = *false* for *i* = 0, 1, ..., *n* - 1.
2. Find an index *i* such that both
 - a. *Finish*[*i*] == *false*
 - b. $Need_i \leq Work$If no such *i* exists, go to step 4.
3. $Work = Work + Allocation_i$
Finish[*i*] = *true*
Go to step 2.
4. If *Finish*[*i*] == *true* for all *i*, then the system is in a safe state.

8.6.3.2 Resource-Request Algorithm

Next, we describe the algorithm for determining whether requests can be safely granted. Let $Request_i$ be the request vector for thread T_i . If $Request_i[j] == k$, then thread T_i wants *k* instances of resource type R_j . When a request for resources is made by thread T_i , the following actions are taken:

1. If $Request_i \leq Need_i$, go to step 2. Otherwise, raise an error condition, since the thread has exceeded its maximum claim.
2. If $Request_i \leq Available$, go to step 3. Otherwise, T_i must wait, since the resources are not available.
3. Have the system pretend to have allocated the requested resources to thread T_i by modifying the state as follows:

$$\begin{aligned} Available &= Available - Request_i \\ Allocation_i &= Allocation_i + Request_i \\ Need_i &= Need_i - Request_i \end{aligned}$$

If the resulting resource-allocation state is safe, the transaction is completed, and thread T_i is allocated its resources. However, if the new state is unsafe, then T_i must wait for $Request_i$, and the old resource-allocation state is restored.

PRACTICE PROBLEMS BASED ON BANKER'S ALGORITHM-

Problem-01:

A single processor system has three resource types X, Y and Z, which are shared by three processes. There are 5 units of each resource type. Consider the following scenario, where the column alloc denotes the number of units of each resource type allocated to each process, and the column request denotes the number of units of each resource type requested by a process in order to complete execution. Which of these processes will finish LAST?

1. P0 2. P1 3. P2
4. None of the above since the system is in a deadlock

	Alloc			Request		
	X	Y	Z	X	Y	Z
P0	1	2	1	1	0	3
P1	2	0	1	0	1	2
P2	2	2	1	1	2	0

Solution- According to question-

$$\bullet \text{Total} = [X \ Y \ Z] = [5 \ 5 \ 5]$$

$$\bullet \text{Total_Alloc} = [X \ Y \ Z] = [5 \ 4 \ 3]$$

Now, Available

$$= \text{Total} - \text{Total_Alloc}$$

$$= [5 \ 5 \ 5] - [5 \ 4 \ 3] = [0 \ 1 \ 2]$$

Step-01:

With the instances available currently, only the requirement of the process P1 can be satisfied.

So, process P1 is allocated the requested resources.

It completes its execution and then free up the instances of resources held by it.

Then, Available

$$= [0 \ 1 \ 2] + [2 \ 0 \ 1]$$

$$= [2 \ 1 \ 3]$$

Step-02:

•With the instances available currently, only the requirement of the process P0 can be satisfied.

•So, process P0 is allocated the requested resources.

•It completes its execution and then free up the instances of resources held by it.

$$\text{Then- Available} = [2 \ 1 \ 3] + [1 \ 2 \ 1] = [3 \ 3 \ 4]$$

Step-03:

- With the instances available currently, the requirement of the process P2 can be satisfied.
- So, process P2 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then- Available

$$= [3 \ 3 \ 4] + [2 \ 2 \ 1]$$

$$= [5 \ 5 \ 5]$$

Thus,

- There exists a safe sequence P1, P0, P2 in which all the processes can be executed.
- So, the system is in a safe state.
- Process P2 will be executed at last.

Thus, Option (C) is correct.

Problem-02:

An operating system uses the banker’s algorithm for deadlock avoidance when managing the allocation of three resource types X, Y and Z to three processes P0, P1 and P2. The table given below presents the current system state. Here, the Allocation matrix shows the current number of resources of each type allocated to each process and the Max matrix shows the maximum number of resources of each type required by each process during its execution.

	Allocation			Max		
	X	Y	Z	X	Y	Z
P0	0	0	1	8	4	3
P1	3	2	0	6	2	0
P2	2	1	1	3	3	3

There are 3 units of type X, 2 units of type Y and 2 units of type Z still available. The system is currently in safe state. Consider the following independent requests for additional resources in the current state-

REQ1: P0 requests 0 units of X, 0 units of Y and 2 units of Z

REQ2: P1 requests 2 units of X, 0 units of Y and 0 units of Z

Which of the following is TRUE?

- 1.Only REQ1 can be permitted
- 2.Only REQ2 can be permitted
- 3.Both REQ1 and REQ2 can be permitted
- 4.Neither REQ1 nor REQ2 can be permitted

Solution-

According to question,

Available = [X Y Z] = [3 2 2]

Now, Need = Max – Allocation

So, we have-

	Allocation			Max			Need		
	X	Y	Z	X	Y	Z	X	Y	Z
P0	0	0	1	8	4	3	8	4	2
P1	3	2	0	6	2	0	3	0	0
P2	2	1	1	3	3	3	1	2	2

Currently, the system is in safe state.

(It is given in question. If we want, we can check)

Checking Whether REQ1 Can Be Entertained-

•Need of P0 = [0 0 2]

•Available = [3 2 2]

Clearly, With the instances available currently, the requirement of REQ1 can be satisfied. So, banker's algorithm assumes that the request REQ1 is entertained. It then modifies its data structures as-

	Allocation			Max			Need		
	X	Y	Z	X	Y	Z	X	Y	Z
P0	0	0	3	8	4	3	8	4	0
P1	3	2	0	6	2	0	3	0	0
P2	2	1	1	3	3	3	1	2	2

Available

= [3 2 2] – [0 0 2] = [3 2 0]

•Now, it follows the safety algorithm to check whether this resulting state is a safe state or not.

•If it is a safe state, then REQ1 can be permitted otherwise not.

Step-01:

- With the instances available currently, only the requirement of the process P1 can be satisfied.
- So, process P1 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then- Available

$$= [3\ 2\ 0] + [3\ 2\ 0] = [6\ 4\ 0]$$

Now, It is not possible to entertain any process.

- The system has entered the deadlock state which is an unsafe state.
- Thus, REQ1 will not be permitted.

Checking Whether REQ2 Can Be Entertained-

Need of P1 = [2 0 0]

Available = [3 2 2]

Clearly,

With the instances available currently, the requirement of REQ1 can be satisfied.

So, banker's algorithm assumes the request REQ2 is entertained.

It then modifies its data structures as-

	Allocation			Max			Need		
	X	Y	Z	X	Y	Z	X	Y	Z
P0	0	0	1	8	4	3	8	4	2
P1	5	2	0	6	2	0	1	0	0
P2	2	1	1	3	3	3	1	2	2

Available

$$= [3\ 2\ 2] - [2\ 0\ 0] = [1\ 2\ 2]$$

- Now, it follows the safety algorithm to check whether this resulting state is a safe state or not.
- If it is a safe state, then REQ2 can be permitted otherwise not.

Step-01:

With the instances available currently, only the requirement of the process P1 can be satisfied.

So, process P1 is allocated the requested resources.

It completes its execution and then free up the instances of resources held by it.

Then- Available

$$= [1\ 2\ 2] + [5\ 2\ 0] = [6\ 4\ 2]$$

Step-02:

- With the instances available currently, only the requirement of the process P2 can be satisfied.
- So, process P2 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then-

Available

$$= [6 \ 4 \ 2] + [2 \ 1 \ 1]$$

$$= [8 \ 5 \ 3]$$

Step-03:

- With the instances available currently, the requirement of the process P0 can be satisfied.
- So, process P0 is allocated the requested resources.
- It completes its execution and then free up the instances of resources held by it.

Then-

Available

$$= [8 \ 5 \ 3] + [0 \ 0 \ 1]$$

$$= [8 \ 5 \ 4]$$

Thus,

- There exists a safe sequence P1, P2, P0 in which all the processes can be executed.
- So, the system is in a safe state.
- Thus, REQ2 can be permitted.

Thus, Correct Option is (B).

Problem-03:

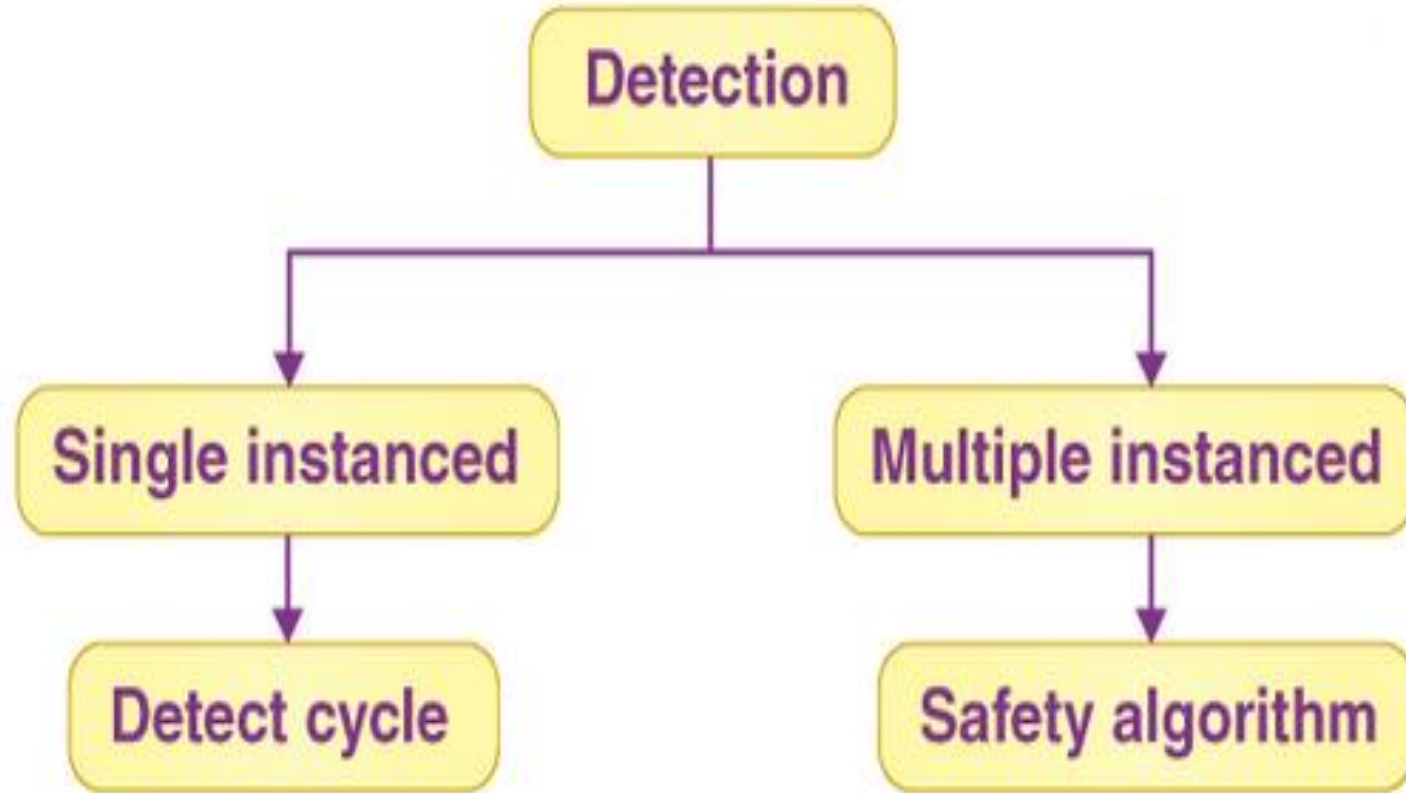
A system has 4 processes and 5 allocatable resource. The current allocation and maximum needs are as follows-

	Allocated					Maximum				
A	1	0	2	1	1	1	1	2	1	3
B	2	0	1	1	0	2	2	2	1	0
C	1	1	0	1	1	2	1	3	1	1
D	1	1	1	1	0	1	1	2	2	0

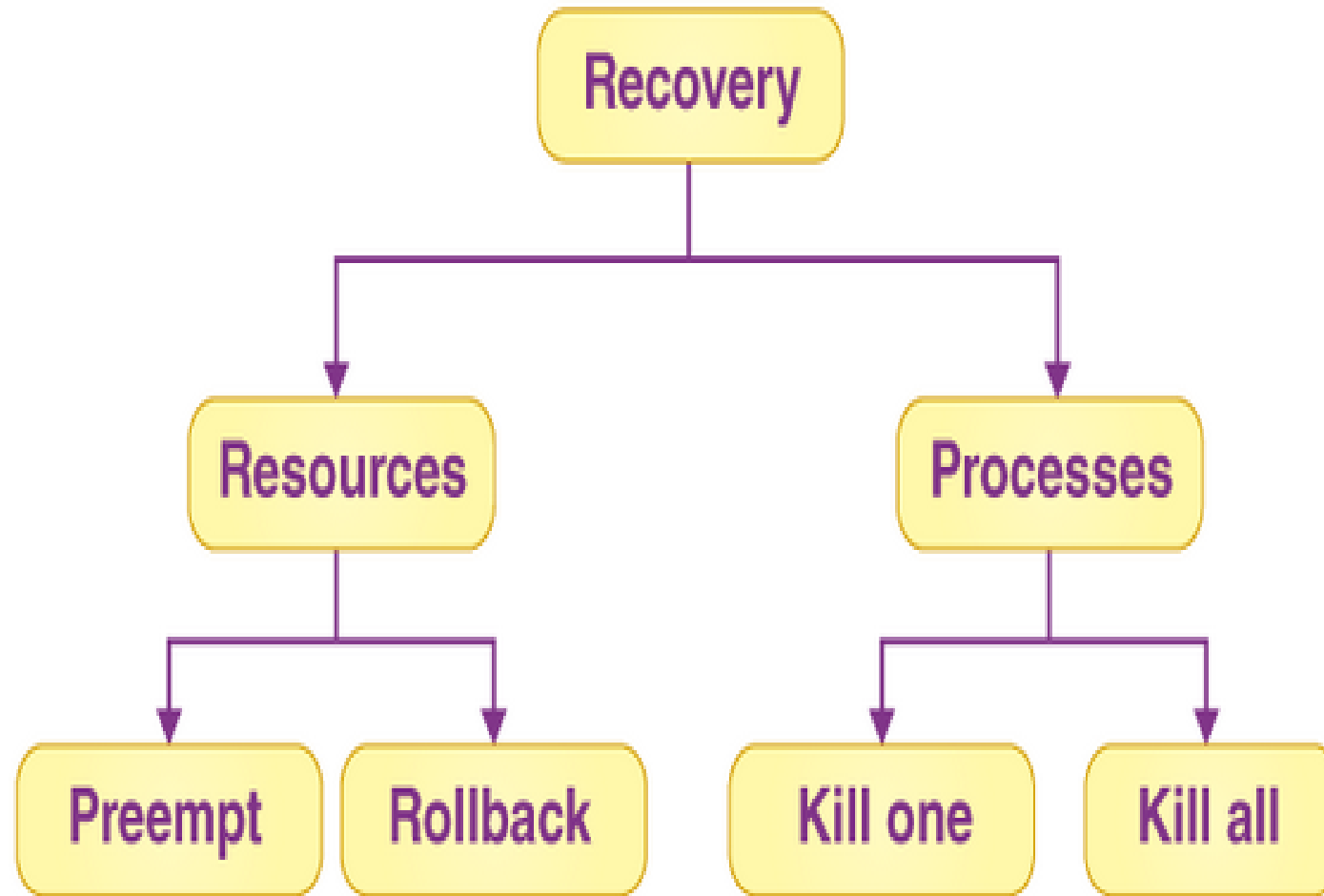
If Available = [0 0 X 1 1], what is the smallest value of x for which this is a safe state?

Home Assignment

Deadlock Detection



Recovery from Deadlock



Deadlock Recovery in Operating System

OS examines either resources or processes to recover the system from deadlocks.

For Resource

Preempt the resource

We can take one of the resources from the resource owner (process) and give it to another process in the hopes that it will finish the execution and release the resource sooner. Choosing a resource that will be snatched will be challenging.

Rollback to a safe state

To reach the deadlock state, the system happens to go through several states. The operating system has the ability to restore the system to a previous safe state. The OS must implement checkpointing at each state for this to work.

When we reach a deadlock, we must reverse all allocations and return to the prior safe state.

For Process

Kill a process

Our problem can be solved by killing a process; however, the bigger issue is deciding which process to kill. A process that has done the least amount of work till now is usually killed by the operating system.

Kill all processes

This is not a persuasive strategy, but it may be used if the problem becomes extremely serious. Killing all processes will result in system inefficiencies because all processes will have to start over.

4. Ostrich Algorithm/Deadlock Ignorance

Because of the ostrich method, the deadlock would be simply disregarded, and it is thought that it would never happen. This is usually done because, in certain systems, the overall cost of dealing with a deadlock is substantially higher than the cost of simply ignoring it because it happens so rarely. As a result, it is simply presumed that the deadlock would never happen, and the system would be rebooted if it did.

Deadlock Ignorance :

Stick your head in the sand and pretend there is no problem at all, this method of solving any problem is called Ostrich Algorithm. This Ostrich algorithm is the most widely used technique in order to ignore the deadlock and also it used for all the single end-users uses. If there is deadlock in the system, then the OS will reboot the system in order to function well. The method of solving any problem varies according to the people.